

Cahier des Charges : Smart Agentic Research Assistant (RAG + Tools + Decision Engine)

1. Introduction

Ce document présente le cahier des charges détaillé pour le développement d'un "Smart Agentic Research Assistant". Ce projet vise à créer un agent IA hybride complet capable de comprendre des requêtes complexes, de prendre des décisions éclairées sur l'utilisation d'outils internes et externes, d'intégrer un système de génération augmentée par la récupération (RAG) et de fournir des réponses contextualisées et précises.

2. Objectifs du Projet

L'objectif principal est de développer un assistant de recherche intelligent qui combine les capacités de compréhension du langage naturel, de raisonnement agentique, d'intégration d'outils et de récupération d'informations pour offrir une expérience utilisateur riche et efficace. Le projet mettra l'accent sur la modularité, la scalabilité et la facilité de déploiement.

3. Fonctionnalités Détaillées

3.1. Compréhension de la Requête

Le système doit être capable d'analyser et d'interpréter diverses requêtes utilisateur, notamment :

- Questions de cours/académiques** : Interrogation sur des documents spécifiques (PDF, articles).

- **Résumé de documents** : Demande de synthèse de contenu (ex: PDF).
- **Recherche d'informations web** : Requêtes nécessitant des données externes ou des recherches en ligne.
- **Explications de concepts** : Demandes de clarification ou de définition.
- **Comparaisons** : Analyse et mise en contraste de plusieurs éléments.

3.2. Module de Décision (Cerveau de l'Agent)

Ce module est le cœur intelligent de l'agent, responsable de la sélection dynamique de l'outil le plus approprié en fonction de l'intention de la requête utilisateur.

Type de Requête	Outil/Module Recommandé
Question sur documents	RAG
Information externe	API / Outil Web
Question simple	LLM direct
Calcul / Logique	Outil interne

3.3. RAG (Retrieval Augmented Generation)

Le module RAG est essentiel pour fournir des réponses contextualisées à partir d'une base de connaissances locale. Il inclut :

- **Ingestion de documents** : Prise en charge de PDF, cours, articles.
- **Chunking** : Division des documents en morceaux de texte gérables.
- **Embeddings** : Génération d'embeddings vectoriels (OpenAI).
- **Base de données vectorielle** : Utilisation de FAISS pour le stockage et la recherche.
- **Récupération et contextualisation** : Intégration des chunks pertinents dans le prompt du LLM.

3.4. Outils Externes (APIs)

L'agent interagit avec des APIs externes :

- **API PubMed** : Pour la recherche médicale scientifique.
- **API OpenFDA** : Pour les informations sur les médicaments.
- **API Wikipedia** : Pour des informations encyclopédiques.
- **API Météo** : Pour des informations météorologiques en temps réel.

3.5. Outils Internes

- **Calculatrice** : Pour des opérations arithmétiques.
- **Interpréteur Python** : Pour des calculs complexes (simulé).
- **Module de résumé** : Pour générer des synthèses de texte.

3.6. Orchestration

Le mécanisme d'orchestration gère le flux de travail :

1. Analyse de la requête.
2. Choix de l'outil via le module de décision.
3. Exécution de l'outil.
4. Combinaison des résultats.
5. Génération de la réponse finale.

3.7. Mémoire (Bonus Fort)

- **Historique utilisateur** : Conservation des interactions précédentes.
- **Contexte conversationnel** : Maintien du fil de la conversation via `ConversationBufferMemory`.

3.8. Interface Utilisateur (Bonus)

- **Technologie** : Streamlit.
- **Fonctionnalité** : Interface de chat pour soumettre des requêtes et afficher les réponses.

4. Architecture du Système

L'architecture suit un flux : User Query -> Intent Analyzer -> Decision Engine -> Tools (RAG/API/Internal) -> Context Builder -> Final Response.

5. Environnement de Développement

- **Système** : Windows / Linux / macOS.
- **Langage** : Python 3.11.
- **Librairies** : `openai`, `langchain`, `faiss-cpu`, `pypdf`, `streamlit`, `requests`.